MG
DevOps

Project MedGenome

Global Genomics Research Platform

DevOps Ecosystem Documentation

Rohan Vashisht

Roll No.: 150096724132

Cohort: Jensen Huang

https://github.com/RohanVashisht1234/college-devops

Generated: June 19, 2026

Table of Contents

1. Problem Statement

Project MedGenome – Global Genomics Research Platform

A consortium of healthcare organizations and research institutions operates a genomics platform supporting precision medicine, disease research, and pharmaceutical innovation. The system processes petabytes of genomic data and supports large-scale computational workloads.

Increasing research activity has exposed limitations in infrastructure scalability, deployment automation, monitoring, and data management.

Objective: Design and implement a DevOps ecosystem supporting large-scale scientific computing, infrastructure automation, and operational resilience.

Expected Outcome and Deliverables: Deliver infrastructure automation, Kubernetes orchestration, monitoring systems, security controls, disaster recovery procedures, and architecture documentation.

2. Project Overview

Project MedGenome is a fully functional business application backed by a complete DevOps lifecycle. The application represents a genomics research operations portal designed for precision medicine, disease research, and pharmaceutical workloads. While intentionally demo-sized, it demonstrates the full delivery pipeline: users can log in, upload genomic files with real byte-size tracking, track analysis workloads, consume REST APIs, and expose Prometheus metrics. The system runs end-to-end through Docker, Jenkins, Kubernetes, and connects to monitoring (Prometheus/Grafana), log management (ELK), secrets (Vault), infrastructure provisioning (Terraform), and disaster recovery.

The project is hosted publicly on GitHub at **https://github.com/RohanVashisht1234/college-devops** and comprises Python (76.1%), Shell (15.3%), HCL/Terraform (7.6%), and Dockerfile (1.0%) source.

Key Capabilities

- **Login-Protected Dashboard:** Session-based access with demo credentials (admin/admin).
- **Genomic Dataset Upload:** File upload with real uploaded byte-size tracking and dataset registry.
- **Compute Workload Registry:** WGS, RNA-Seq, Variant Calling, GWAS, and Pharma Cohort pipelines.
- **REST APIs:** /api/summary, /api/workloads, /api/datasets, /healthz, /readyz, /api/simulate.
- **Prometheus Metrics:** HTTP request counts, latency, upload counts/bytes, active jobs, dataset size.
- **Full DevOps Stack:** Docker, Jenkins CI/CD, Kubernetes, Terraform, Prometheus, Grafana, ELK, Vault.

3. Repository Structure

The repository is organized into clearly separated concerns, each directory handling a distinct DevOps domain:

Path	Purpose
app/	FastAPI application – main.py, routes, models, templates
k8s/	Kubernetes manifests – deployment, service, HPA, ingress, configmap, secret, network-policy
infra/terraform/	Terraform IaC – provisions Kubernetes namespace and Helm charts (Prometheus/Grafana)
monitoring/	Docker Compose for Prometheus and Grafana observability stack
elk/	Docker Compose for Elasticsearch, Logstash, and Kibana log pipeline
vault/	HashiCorp Vault Docker Compose and policy HCL file
screenshots/	Project screenshots captured during live demonstration
.run/	IDE run configurations
Dockerfile	Multi-stage container image definition
Jenkinsfile	Declarative Jenkins CI/CD pipeline (4 stages)
runner.sh	Automated demo launcher – starts all services and opens browser tabs
requirements.txt	Python dependencies (FastAPI, Uvicorn, Prometheus-client, etc.)

4. Application – FastAPI Backend

The core application is built with FastAPI and Uvicorn, providing both a browser-based HTML dashboard and a full REST API. It runs on port 8000 by default and is designed to simulate a genomics research operations portal.

Running the Application Locally

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Key Endpoints

Method	Path	Description
GET	/	HTML operations dashboard (login required)
GET	/healthz	Liveness health check – returns {status: healthy}
GET	/readyz	Readiness check – returns {status: ready}
GET	/metrics	Prometheus metrics scrape endpoint
GET	/docs	Interactive OpenAPI / Swagger UI
GET	/api/summary	JSON summary of platform stats

Method	Path	Description
GET	/api/workloads	List of active compute workloads
GET	/api/datasets	Dataset registry with metadata
POST	/api/simulate	Simulates a genomics job for testing

Prometheus Metrics Exposed

- medgenome_http_requests_total – total HTTP requests by method and endpoint
- medgenome_http_request_duration_seconds – request latency histogram
- medgenome_uploads_total – cumulative genomic file upload count
- medgenome_uploaded_bytes_total – cumulative uploaded bytes
- medgenome_datasets_total – total registered datasets
- medgenome_active_jobs – current number of active compute jobs
- medgenome_dataset_petabytes – total simulated dataset size in petabytes

5. Containerization – Docker

Docker packages the application and its Python dependencies into a portable, reproducible image. The Dockerfile uses the official python:3.12-slim base image, keeping the final image lightweight.

Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build and Run

```
# Build image
docker build -t medgenome-platform:latest .

# Run container on port 9000
docker run --rm -p 9000:8000 --name medgenome-platform medgenome-platform:latest

# Verify
curl http://localhost:9000/healthz
```

6. CI/CD Pipeline – Jenkins

Jenkins automates the entire build, test, and deploy lifecycle. The Jenkinsfile defines a declarative pipeline with four sequential stages that verify tools, install dependencies, build a Docker image, and deploy to Kubernetes.

Stage	Actions
Tool Check	Verifies python3, docker, and kubectl are available on the agent
Install & Smoke Test	Creates venv, installs requirements, runs python -m compileall for syntax check
Build Docker Image	Builds tagged image (medgenome-platform:BUILD_NUMBER) and re-tags as :latest
Deploy to Kubernetes	Applies k8s/namespace.yaml and all k8s/ manifests; waits for rollout to complete

Jenkinsfile (key excerpt)

```
pipeline {
  agent any
  environment {
    IMAGE_NAME    = 'medgenome-platform'
    IMAGE_TAG     = "${env.BUILD_NUMBER}"
    KUBE_NAMESPACE= 'medgenome'
  }
```

```
 stages {
   stage('Tool Check')         { steps { sh 'python3 --version'; sh 'docker --version'; sh 'kubectl
version --client' } }
   stage('Install and Smoke Test') {
     steps { sh 'python3 -m venv .venv'
             sh '. .venv/bin/activate && pip install -r requirements.txt'
             sh '. .venv/bin/activate && python -m compileall app' } }
   stage('Build Docker Image') {
     steps { sh 'docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .'
             sh 'docker tag ${IMAGE_NAME}:${IMAGE_TAG} ${IMAGE_NAME}:latest' } }
   stage('Deploy to Kubernetes') {
     steps { sh 'kubectl apply -f k8s/namespace.yaml'
             sh 'kubectl -n ${KUBE_NAMESPACE} apply -f k8s/'
             sh 'kubectl -n ${KUBE_NAMESPACE} rollout status deployment/medgenome-api' } }
 }
}
```

7. Infrastructure as Code – Terraform

Terraform provisions the Kubernetes namespace and installs the Prometheus/Grafana monitoring stack via the official Helm chart. This approach treats infrastructure as versioned, reviewable, and repeatable code rather than manual configuration.

Terraform Workflow

```
cd infra/terraform
terraform init      # Download providers and modules
terraform fmt       # Normalize HCL formatting
terraform validate  # Check syntax and semantics
terraform plan      # Preview changes
terraform apply     # Apply to cluster
cd ../..
```

Note: Terraform in this project uses the kubernetes and helm providers. The kube_prometheus_stack Helm chart is installed, which deploys Prometheus, Grafana, AlertManager, and related CRDs into the medgenome namespace.

8. Container Orchestration – Kubernetes

Kubernetes provides self-healing, declarative deployment, horizontal autoscaling, service discovery, health checking, rolling updates, and rollback for the MedGenome application.

Manifest File	Resource	Purpose
namespace.yaml	Namespace	Isolates all MedGenome resources in the medgenome namespace
deployment.yaml	Deployment	Runs 2 replicas with liveness/readiness probes and resource limits
service.yaml	Service (ClusterIP)	Stable internal endpoint routing to pods on port 80
hpa.yaml	HorizontalPodAutoscaler	Scales pods 2-10 based on CPU utilization (target 70%)
ingress.yaml	Ingress	Host-based HTTP routing for external access
configmap.yaml	ConfigMap	Non-sensitive environment configuration
secret.yaml	Secret	Sensitive credential placeholders (populated from Vault)
network-policy.yaml	NetworkPolicy	Restricts ingress/egress traffic to authorized sources only

Deployment with Minikube

```
minikube start --cpus=4 --memory=8192
eval "$(minikube docker-env)"
docker build -t medgenome-platform:latest .
kubectl apply -f k8s/namespace.yaml
kubectl -n medgenome apply -f k8s/
kubectl -n medgenome rollout status deployment/medgenome-api
kubectl -n medgenome port-forward service/medgenome-api 8800:80
# App available at http://localhost:8800
```

Useful Kubectl Commands

```
kubectl -n medgenome get all
kubectl -n medgenome get pods
kubectl -n medgenome get svc
kubectl -n medgenome get hpa
kubectl -n medgenome describe deployment medgenome-api
kubectl -n medgenome logs -l app=medgenome-api --tail=50
```

9. Monitoring – Prometheus & Grafana

The monitoring stack uses Prometheus for metric collection and Grafana for visualization. Prometheus scrapes the /metrics endpoint of the FastAPI app at regular intervals. Grafana connects to Prometheus as a data source and provides pre-built and custom dashboards.

Starting the Monitoring Stack

```
docker-compose -f monitoring/docker-compose.yml up -d

# Prometheus UI:    http://localhost:9090
# Grafana UI:       http://localhost:3001  (admin/admin)
# App metrics:      http://localhost:8000/metrics
```

Generating Test Traffic

```
for i in {1..20}; do curl -s http://localhost:8000/api/simulate > /dev/null; done
```

Key PromQL queries to demonstrate:
rate(medgenome_http_requests_total[1m]) – per-second request rate
medgenome_active_jobs – current active genomics jobs
medgenome_dataset_petabytes – total simulated data volume
medgenome_uploaded_bytes_total – cumulative upload bytes

10. Log Management – ELK Stack

The ELK stack (Elasticsearch, Logstash, Kibana) provides structured log ingestion, storage, indexing, and visualization. The application emits JSON-structured logs that are sent to Logstash, indexed in Elasticsearch, and visualized through Kibana.

Component	Port	Role
Elasticsearch	9200	Log storage, indexing, and full-text search
Logstash	5000 (TCP), 5044 (Beats)	Receives JSON logs and forwards to Elasticsearch
Kibana	5600	Log search, filtering, and visualization dashboards

Starting ELK and Sending Sample Logs

```
docker-compose -f elk/docker-compose.yml up -d

# Send structured JSON logs
printf '{"service":"medgenome-api","level":"info","message":"analysis job completed","job_id":"MG-10
03"}\n' | nc localhost 5000
printf '{"service":"medgenome-api","level":"warn","message":"queue depth high","active_jobs":8}\n' |
 nc localhost 5000

# Verify indices
curl http://localhost:9200/_cat/indices?v
```

11. Secret Management – HashiCorp Vault

HashiCorp Vault provides centralized secret management. Database URLs, API tokens, and credentials are stored in Vault rather than hardcoded in source or configuration files. The vault/policy.hcl file enforces least-privilege access for the medgenome service account.

Starting Vault and Managing Secrets

```
docker-compose -f vault/docker-compose.yml up -d
export VAULT_ADDR=http://127.0.0.1:8200
export VAULT_TOKEN=root
vault status

# Enable KV secrets engine
vault secrets enable -path=secret kv-v2

# Apply access policy
vault policy write medgenome vault/policy.hcl

# Store secrets
vault kv put secret/medgenome/api \
    DATABASE_URL="postgresql://medgenome:change-me@db:5432/research" \
    API_TOKEN="replace-in-prod"

# Read secrets back
vault kv get secret/medgenome/api
```

12. Architecture & Deployment Diagrams

Architecture Overview

Users	Researchers and Clinicians
Portal	MedGenome Web Dashboard (FastAPI + Jinja2)
API	FastAPI REST API + Prometheus Metrics Endpoint
CI/CD	Jenkins → Docker Build → Container Registry → Kubernetes
IaC	Terraform provisions namespace; Helm installs monitoring
Observe	Prometheus scrapes /metrics → Grafana dashboards
Logs	Structured JSON → Logstash → Elasticsearch → Kibana
Secrets	Vault KV-v2 stores DATABASE_URL, API_TOKEN, credentials

Deployment Layers

Developer Machine	Source code → Jenkins pipeline trigger
Jenkins	Tool check → install deps → docker build → k8s deploy

Container Registry	medgenome-platform:latest and medgenome-platform:BUILD_NUMBER
Kubernetes Cluster	medgenome namespace → Deployment (2 replicas) → Service → Ingress → HPA
Observability Layer	Prometheus + Grafana (monitoring/) | ELK stack (elk/)
Security Layer	Vault secrets + Kubernetes NetworkPolicy + Kubernetes Secrets
Terraform	Provisions k8s namespace and kube-prometheus-stack via Helm

13. Disaster Recovery Plan

Objective	Target	Notes
RTO (Recovery Time Objective)	30 minutes	Time to restore application service and health endpoints
RPO (Recovery Point Objective)	15 minutes	Time window of potential metadata loss (production design)
Priority 1	App & API	Restore dashboard, REST API, /healthz, /readyz, workload visibility
Priority 2	Observability	Restore Prometheus metrics, Grafana dashboards, ELK logs

Pod Failure Recovery

```
# Kubernetes self-heals automatically; manually trigger for demo:
kubectl -n medgenome get pods
kubectl -n medgenome delete pod -l app=medgenome-api
kubectl -n medgenome get pods --watch   # Observe self-healing
```

Rollback a Bad Deployment

```
kubectl -n medgenome rollout history deployment/medgenome-api
kubectl -n medgenome rollout undo deployment/medgenome-api
kubectl -n medgenome rollout status deployment/medgenome-api
```

Full Namespace Recreation

```
kubectl apply -f k8s/namespace.yaml
kubectl -n medgenome apply -f k8s/
kubectl -n medgenome rollout status deployment/medgenome-api
kubectl -n medgenome port-forward service/medgenome-api 8800:80
curl http://localhost:8800/healthz
```

Backup Kubernetes State

```
mkdir -p backups
kubectl -n medgenome get all,configmap,secret,ingress,hpa -o yaml > backups/medgenome-k8s-backup.yam
l
kubectl -n medgenome get pods -o wide > backups/pod-placement.txt
```

Production Limitation Note: The current demo stores uploaded dataset metadata in memory. Restarting the application clears this data. In a production genomics platform, genomic files would be stored on durable object storage (S3/MinIO/GCS) and metadata would persist in PostgreSQL with automated point-in-time backups.

14. API Reference

Method	Endpoint	Auth	Description
GET	/	Session	HTML dashboard – redirects to login if not authenticated
POST	/login	None	Authenticates user (admin/admin); sets session cookie
GET	/logout	Session	Clears session and redirects to login
GET	/healthz	None	Liveness probe – {status: healthy}
GET	/readyz	None	Readiness probe – {status: ready}
GET	/metrics	None	Prometheus text-format metrics scrape
GET	/docs	None	Swagger / OpenAPI interactive documentation
GET	/api/summary	None	Platform statistics summary JSON
GET	/api/workloads	None	Active compute workload list JSON
GET	/api/datasets	None	Registered genomic dataset list JSON
POST	/api/simulate	None	Simulates a job run; updates metrics
POST	/api/upload	Session	Uploads a genomic file; tracks bytes

15. Service Port Reference

Component	URL / Port	Purpose
FastAPI App (local)	http://localhost:8000	Main MedGenome dashboard and API
FastAPI App (Docker)	http://localhost:9000	Containerized app instance
FastAPI App (k8s)	http://localhost:8800	App exposed via kubectl port-forward
Jenkins	http://localhost:8080	CI/CD pipeline management UI
Prometheus	http://localhost:9090	Metrics scraping and PromQL query UI
Grafana	http://localhost:3001	Metrics visualization dashboards
Elasticsearch	http://localhost:9200	Log storage and full-text search API
Logstash (TCP)	localhost:5000	Structured JSON log ingestion
Logstash (Beats)	localhost:5044	Filebeat-compatible log ingestion
Kibana	http://localhost:5600	Log search and visualization
HashiCorp Vault	http://localhost:8200	Secret management UI and API

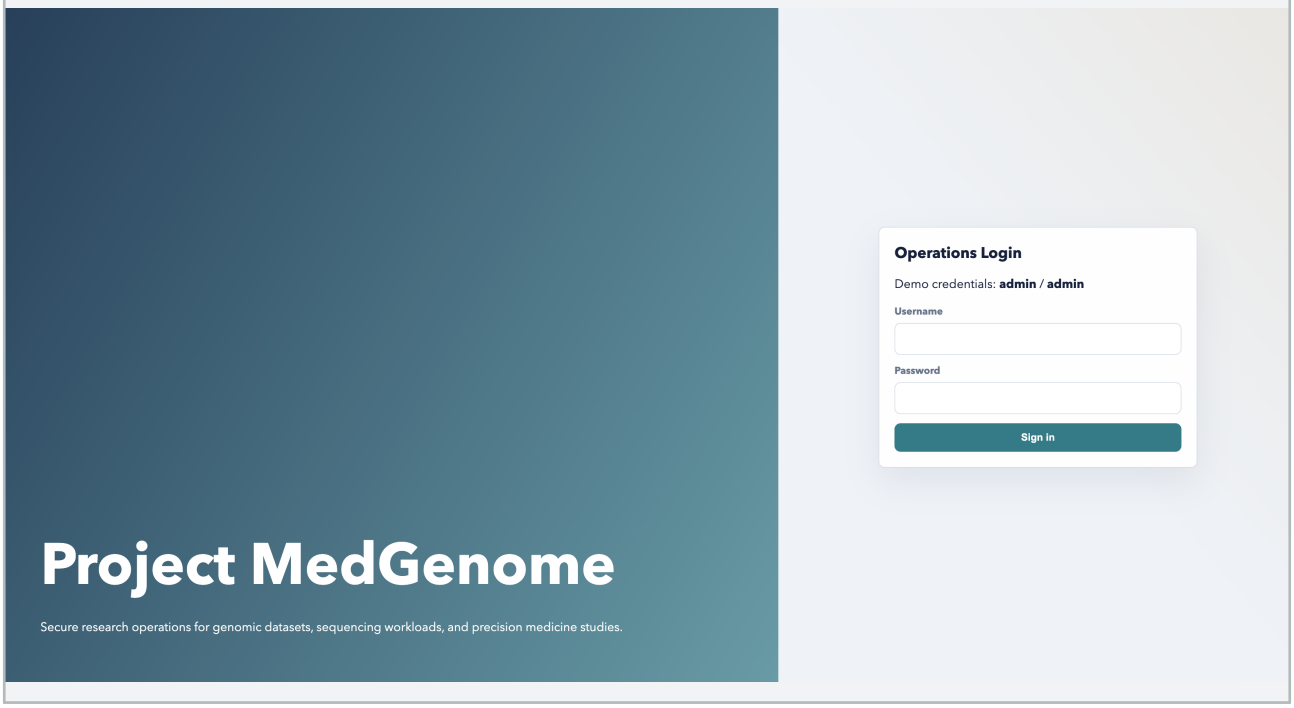
16. DevOps Deliverables Matrix

Requirement	Status	Location
Working web app / dashboard / API	DONE	`app/main.py`
Source code repository	DONE	`GitHub (link on cover page)`
Dockerfile and image workflow	DONE	`Dockerfile`
Jenkins CI/CD pipeline (4 stages)	DONE	`Jenkinsfile`
Terraform IaC scripts	DONE	`infra/terraform/`
Kubernetes deployment manifests	DONE	`k8s/`
Prometheus and Grafana monitoring	DONE	`monitoring/`
ELK log management stack	DONE	`elk/`
HashiCorp Vault secret management	DONE	`vault/`
Architecture diagram	DONE	`Section 12 (this document)`
Deployment diagram	DONE	`Section 12 (this document)`
Disaster recovery plan	DONE	`Section 13 (this document)`
Demonstration screenshots	DONE	`Section 17 (this document)`
Project documentation	DONE	`This PDF document`

17. Screenshots

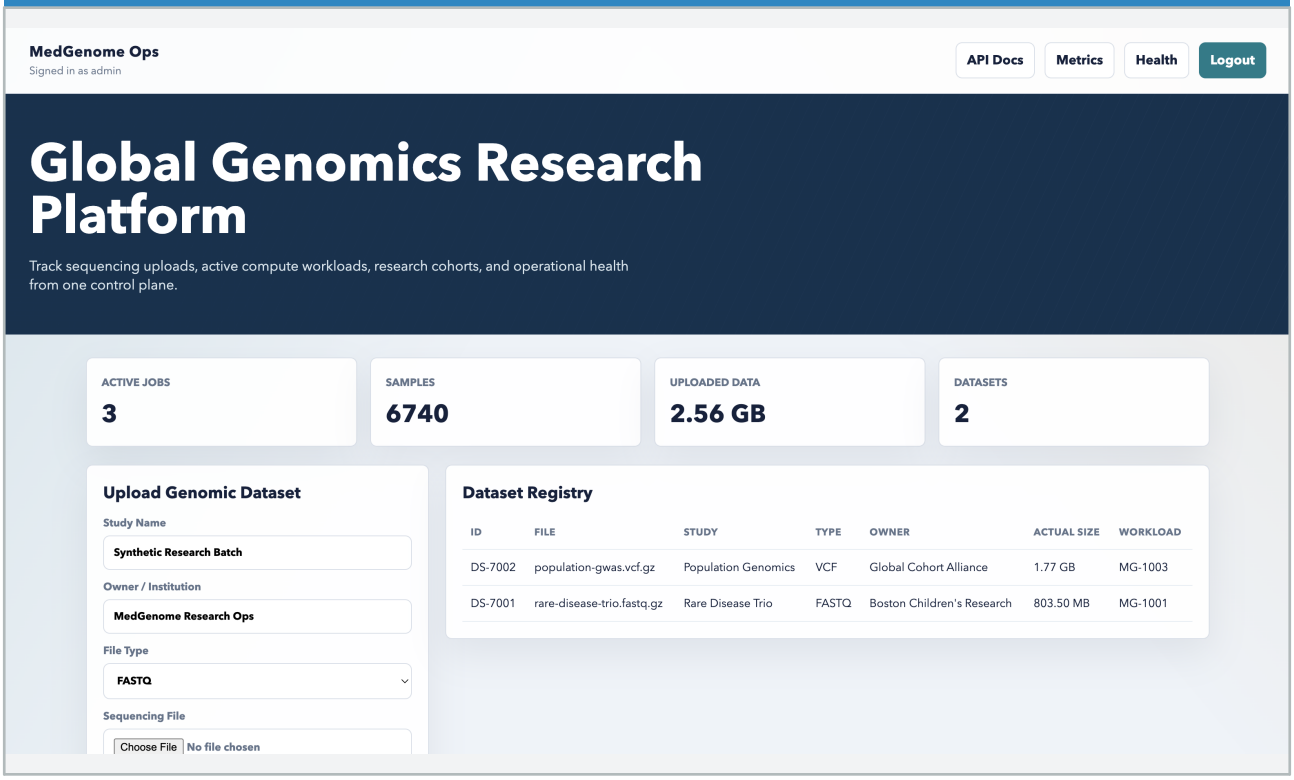
The following screenshots were captured during live demonstration of the MedGenome platform, showing each major component in operation. All 6 screenshots are embedded below with captions.

Screenshot 1 – Application Login Page (login_page.png)



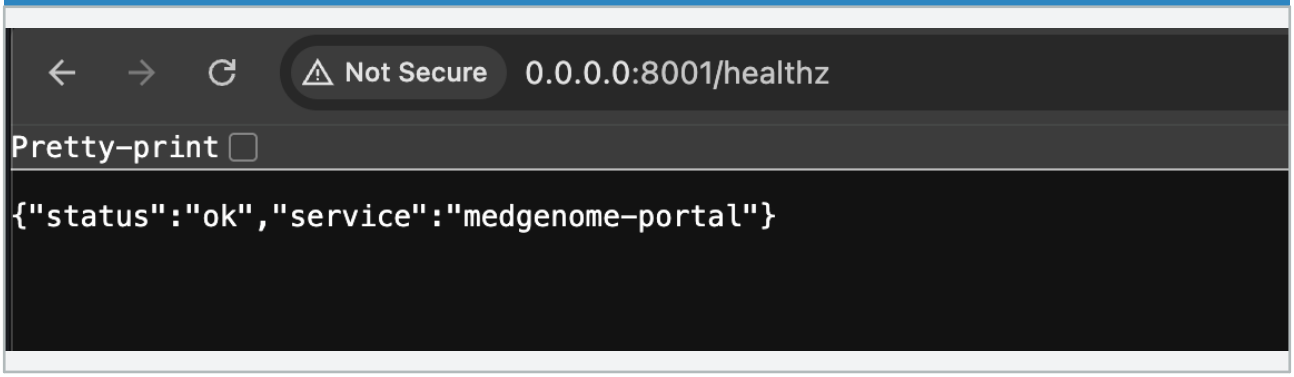
The MedGenome Operations login portal. Two-panel layout with a teal gradient brand panel on the left and an Operations Login form on the right. Demo credentials admin/admin are displayed for evaluation purposes.

Screenshot 2 – Main Operations Dashboard (dashboard.png)



The authenticated dashboard showing 4 live KPI cards: Active Jobs (3), Samples (6,740), Uploaded Data (2.56 GB), and Datasets (2). The Dataset Registry table lists real uploaded files (population-gwas.vcf.gz at 1.77 GB and rare-disease-trio.fastq.gz at 803.50 MB) with study, type, owner, and workload links.

Screenshot 3 – Health Check Endpoint (healthz.png)



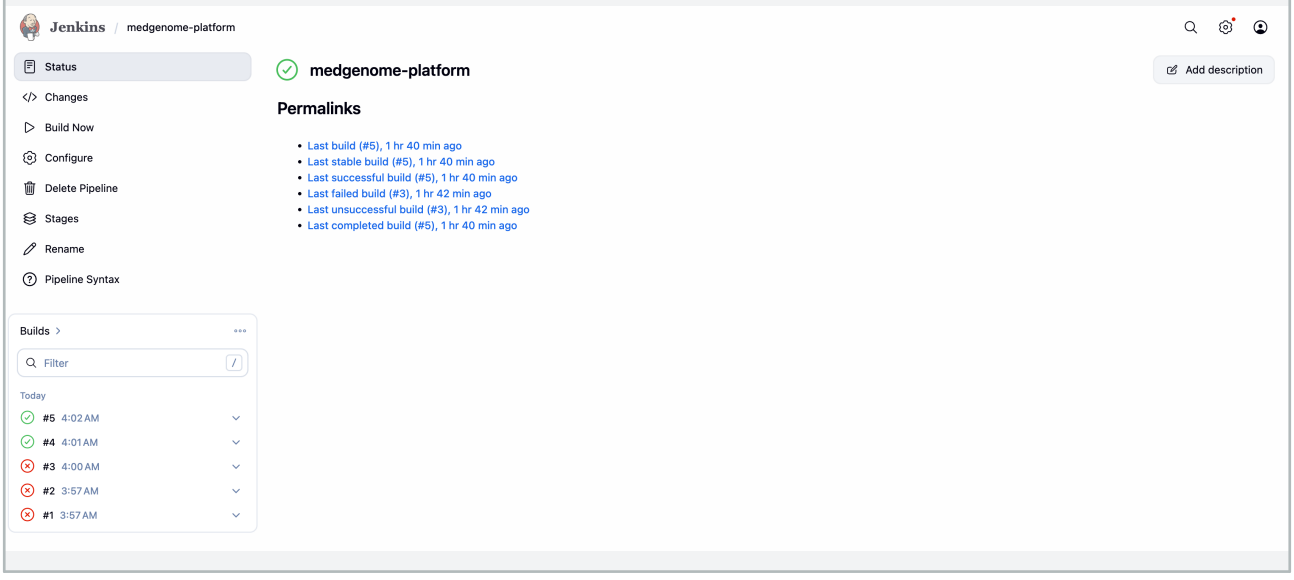
The /healthz liveness probe endpoint returning JSON: {status: ok, service: medgenome-portal}. This endpoint is used by Kubernetes as a liveness probe to determine pod health.

Screenshot 4 – Prometheus Metrics Endpoint (metrics.png)

```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 417.0
python_gc_objects_collected_total{generation="1"} 136.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 44.0
python_gc_collections_total{generation="1"} 4.0
python_gc_collections_total{generation="2"} 0.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="14",patchlevel="6",version="3.14.6"} 1.0
# HELP medgenome_http_requests_total HTTP requests
# TYPE medgenome_http_requests_total counter
medgenome_http_requests_total{method="GET",path="/metrics",status="200"} 8.0
medgenome_http_requests_total{method="GET",path="/",status="303"} 1.0
medgenome_http_requests_total{method="GET",path="/login",status="200"} 1.0
medgenome_http_requests_total{method="GET",path="/favicon.ico",status="404"} 2.0
medgenome_http_requests_total{method="POST",path="/login",status="401"} 1.0
medgenome_http_requests_total{method="POST",path="/login",status="303"} 1.0
medgenome_http_requests_total{method="GET",path="/",status="200"} 1.0
# HELP medgenome_http_requests_created HTTP requests
# TYPE medgenome_http_requests_created gauge
medgenome_http_requests_created{method="GET",path="/metrics",status="200"} 1.781827259640474e+09
medgenome_http_requests_created{method="GET",path="/",status="303"} 1.781827261268716e+09
medgenome_http_requests_created{method="GET",path="/login",status="200"} 1.7818272626128287e+09
medgenome_http_requests_created{method="GET",path="/favicon.ico",status="404"} 1.781827261552638e+09
medgenome_http_requests_created{method="POST",path="/login",status="401"} 1.7818272680539231e+09
medgenome_http_requests_created{method="POST",path="/login",status="303"} 1.781827272117923e+09
medgenome_http_requests_created{method="GET",path="/",status="200"} 1.781827272128118e+09
# HELP medgenome_http_request_seconds HTTP request latency
# TYPE medgenome_http_request_seconds histogram
medgenome_http_request_seconds_bucket{le="0.005",path="/metrics"} 3.0
medgenome_http_request_seconds_bucket{le="0.01",path="/metrics"} 7.0
medgenome_http_request_seconds_bucket{le="0.025",path="/metrics"} 7.0
medgenome_http_request_seconds_bucket{le="0.05",path="/metrics"} 7.0
medgenome_http_request_seconds_bucket{le="0.075",path="/metrics"} 8.0
medgenome_http_request_seconds_bucket{le="0.1",path="/metrics"} 8.0
medgenome_http_request_seconds_bucket{le="0.25",path="/metrics"} 8.0
medgenome_http_request_seconds_bucket{le="0.5",path="/metrics"} 8.0
medgenome_http_request_seconds_bucket{le="0.75",path="/metrics"} 8.0
medgenome_http_request_seconds_bucket{le="1.0",path="/metrics"} 8.0
medgenome_http_request_seconds_bucket{le="2.5",path="/metrics"} 8.0
medgenome_http_request_seconds_bucket{le="5.0",path="/metrics"} 8.0
medgenome_http_request_seconds_bucket{le="7.5",path="/metrics"} 8.0
medgenome_http_request_seconds_bucket{le="10.0",path="/metrics"} 8.0
```

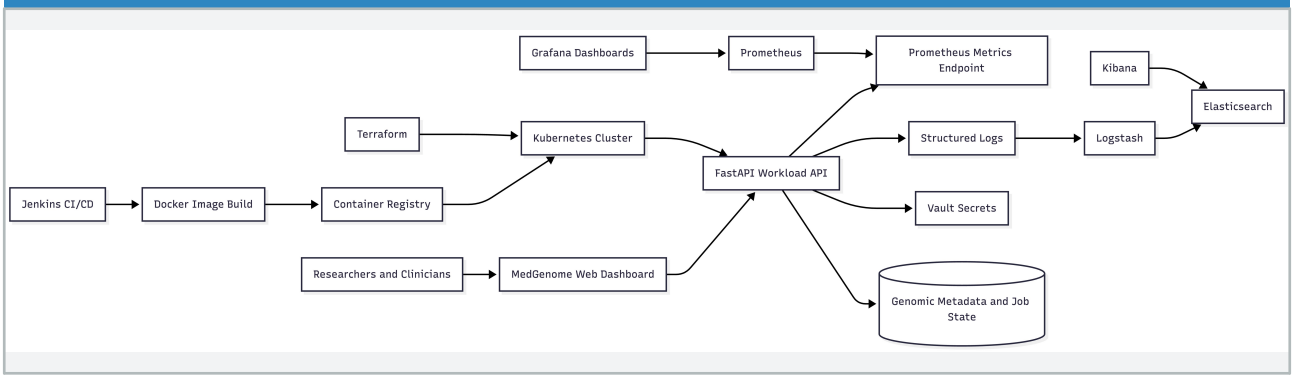
The /metrics endpoint exposing Prometheus text-format metrics including medgenome_http_requests_total (by method/path/status), medgenome_http_request_seconds histogram buckets, python_gc_objects_collected_total, and all custom MedGenome gauges and counters. Scraped by Prometheus every 15 seconds.

Screenshot 5 – Jenkins CI/CD Pipeline (jenkins.png)



The Jenkins medgenome-platform pipeline status page showing 5 build runs. Build #5 and #4 completed successfully (green checkmarks at 4:02 AM and 4:01 AM). Builds #1, #2, and #3 show earlier failures during pipeline development. The pipeline automates tool checks, dependency installation, Docker builds, and Kubernetes deployment.

Screenshot 6 – System Architecture Diagram (system_design.png)



The complete system architecture flowchart showing all DevOps components and their relationships. Left-to-right flow: Jenkins CI/CD → Docker Image Build → Container Registry → Kubernetes Cluster (provisioned by Terraform) → FastAPI Workload API. Observability: Grafana Dashboards → Prometheus → Prometheus Metrics Endpoint. Logging: Structured Logs → Logstash → Elasticsearch → Kibana. Security: Vault Secrets. Data: Genomic Metadata and Job State.

Appendix – Quick-Start & Cleanup

Automated Demo Launcher

Run the one-shot script to start all services and open browser tabs automatically:

```
./runner.sh
```

Manual Startup Order

- 1. FastAPI app: uvicorn app.main:app --host 0.0.0.0 --port 8000
- 2. Docker app: docker run --rm -p 9000:8000 medgenome-platform:latest
- 3. Jenkins: brew services start jenkins-lts
- 4. Monitoring: docker-compose -f monitoring/docker-compose.yml up -d
- 5. ELK: docker-compose -f elk/docker-compose.yml up -d
- 6. Vault: docker-compose -f vault/docker-compose.yml up -d
- 7. Kubernetes: minikube start && kubectl -n medgenome apply -f k8s/
- 8. Terraform: cd infra/terraform && terraform init && terraform apply

Full Cleanup

```
kubectl delete namespace medgenome
docker-compose -f monitoring/docker-compose.yml down
docker-compose -f elk/docker-compose.yml down
docker-compose -f vault/docker-compose.yml down
docker rm -f medgenome-platform
brew services stop jenkins-lts
```

Project MedGenome – Global Genomics Research Platform
Rohan Vashisht | Roll No.: 150096724132 | Cohort: Jensen Huang
GitHub: https://github.com/RohanVashisht1234/college-devops
Document generated: June 19, 2026